Difference-Proportional Verification of Live Heterogeneous Database Migrations with Invertible Bloom Lookup Tables

Nithin Narla
Independent Researcher
nithin.nnv@gmail.com

Suhas Jangoan
Independent Researcher
suhas.jangoan@ieee.org

Ayan Putatunda
Independent Researcher
ayanputatunda87@gmail.com

Abstract*—Migrating a live system from one database to another, for example from a relational store to a wide-column NoSQL store, requires evidence that the two stores hold identical data. Existing tools compare full scans or checksum key ranges and bisect mismatches. Their cost grows with table size, they assume both stores scan in the same key order, and they report false differences while writes are in flight. We present DELTAPROOF, a verifier whose communication cost grows with the number of differences rather than the size of the data. Each store maintains partitioned invertible Bloom lookup tables (IBLTs) over a canonical encoding of its records. Subtracting the two sketches and decoding the result yields the exact differing keys in one round, independent of key order. A watermark and confirmation protocol separates replication lag from true divergence, and a ladder of sketch sizes chosen by a strata estimator avoids decoding failures. On up to 10^7 ledger entries with eight injected fault classes, DELTAPROOF found every injected difference with no false positives. At 10^7 rows and about 1,100 differences it transferred 0.78 MB and read no table rows, while Merkle trees transferred 32 MB, range-checksum bisection 280 MB and a full scan 480 MB, each reading 20–52 million rows. Between real PostgreSQL 16 and Cassandra 4.1 stores, DELTAPROOF found all 120 injected faults; a digest without canonical encoding falsely flagged 99.9% of rows. Under a live TPC-C workload replicated from PostgreSQL to Cassandra through a change stream, sketches maintained from the stream stayed exact, verification ran every 10 s, version-aware confirmation made no false report in 38 checks, including with out-of-order replication, and the final answers were exact. Against PinSketch, DELTAPROOF sends 3–9× more bytes but decodes 5–10× faster beyond 10^4 differences and is cheaper to maintain per write.

***Index Terms*—data migration, set reconciliation, invertible Bloom lookup tables, data verification, heterogeneous databases, consistency**

I. INTRODUCTION

A live migration is complete only when the source and target stores provably hold the same data. For financial ledgers, payments and inventory, one missing or altered record is a correctness failure. Live migrations combine dual writes or change data capture (CDC), a backfill of historical data and a final cutover [1], [2]. Verification is the weak step. Current practice falls into three groups:

- *Full scans* read every record from both stores. Cost grows with table size and loads production databases.
- *Range-checksum bisection* [3] hashes key ranges in both stores and splits mismatched ranges. It needs aligned key order, which hash-partitioned stores such as Cassandra [4] do not provide, and several round trips.
- *Merkle-tree anti-entropy* [4], [5] compares replicas of one database with identical encodings and repairs whole segments.

None targets the common case of a heterogeneous migration: many records, few differences, different storage orders and encodings, and writes still arriving.

We observe that this is a *set reconciliation* problem. IBLTs [6], [7] reconcile sets with communication proportional to the size of the difference, in one round and independent of order. IBLTs are also linear, so sketches can be maintained from a change stream instead of rebuilt by scanning. We contribute:

1) a canonical record encoding that hashes logically equal records identically across relational and wide-column stores (Section III-A);
2) partitioned, incrementally maintained IBLT sketches with a size ladder selected by a strata estimator (Section III);
3) a watermark and confirmation protocol for verification while writes are in flight (Section IV);
4) an open-source implementation and an evaluation against full scans, bisection, Merkle trees, aggregates and PinSketch, in process up to 10^7 rows, on TPC-H data, and under a live TPC-C workload replicated from PostgreSQL to Cassandra (Sections VI–VII).

II. BACKGROUND

A. Live migration model

The source store S stays authoritative. A dual-write or CDC path copies new writes to the target T and a backfill copies history. Verification must show $S = T$ before reads and then writes move to T. Records have a primary key k, a value v and a source commit version.

B. Divergence classes

Table I lists the faults a verifier must catch; each becomes an injected fault in Section VII. Offsetting errors, such as one

PRELIMINARY DRAFT

TABLE I
DIVERGENCE CLASSES AND THEIR EFFECT ON THE TARGET

Fault	Example cause	Effect on T
Lost write	Secondary write fails silently	Key missing
Phantom	Retry without idempotency	Extra key
Stale value	Backfill overwrites newer write	Old value
Corruption	Type-mapping bug	Wrong value
Precision	Decimal $\rightarrow$ float rounding	Amount off by 0.01
Time zone	Timestamp without zone	Time off by hours
Lost delete	Delete not propagated	Extra key
Offsetting	Lost write + equal phantom	Aggregates unchanged

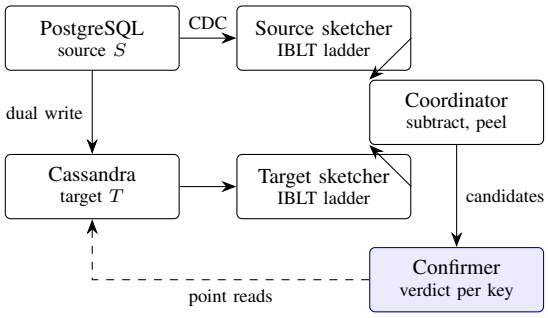


Fig. 1. DELTAPROOF architecture. Only sketches cross between the stores; individual records are read only for candidate keys.

lost and one duplicated entry of equal amount, leave counts and sums unchanged and defeat aggregate checks.

C. Invertible Bloom lookup tables

An IBLT [6] is an array of m cells; each element x is hashed into q cells (we use $q = 3$ subtables). A cell holds a count, an XOR of keys and an XOR of checksums:

$$\text{count} \mathrel{+}= 1, \quad \text{keySum} \oplus= h(x), \quad \text{checkSum} \oplus= g(x). \tag{1}$$

Two sketches built with the same hashes subtract cell by cell; common elements cancel, leaving the symmetric difference. *Peeling* recovers it: a cell with count ± 1 and a valid checksum holds one element, which is removed from its other cells. Decoding succeeds with high probability when m exceeds about 1.23 times the number of differences d for $q = 3$ [6], [7]. Strata estimators [7] and rateless designs [8] handle unknown d.

III. DELTAPROOF DESIGN

A. Canonical record encoding

Each record becomes an element $e = (k, \delta)$ with a 128-bit key and a 64-bit digest $\delta = H_{64}(\text{canon}(v))$ of the canonical value. The canonical form fixes the column order through the migration's schema mapping, and normalizes each type: decimals become exact scaled integers at a declared scale; timestamps become UTC at a declared unit (milliseconds when one store keeps only milliseconds); text is Unicode NFC; `CHAR(n)` padding is dropped; floats are compared by IEEE-754 bits with -0.0 and NaN folded; nulls carry their own tag, so $\text{null} \neq \texttt{""} \neq 0$. Composite keys of up to four integer parts are packed with an 8-bit table tag into 128 bits, so one sketch covers many tables. A changed value appears as two elements with the same key and is reported as a *value mismatch*.

B. Partitioned sketches and the size ladder

Records are assigned to P partitions by a hash of the key, independent of physical order. Each partition is an IBLT; a partition that fails to decode is the only one rescanned. Because update cost is linear, each store keeps a *ladder* of levels at once, with capacities $C_i = 64 \cdot 8^i$ up to $n/8$, $P_i = \max(1, C_i/256)$ partitions and $2C_i/P_i$ cells each. Verification sends a 32-stratum estimator first. If every stratum decodes, the difference is already exact; otherwise the coordinator decodes the smallest level with $C_i \geq 1.25\,\hat{d}$. Inserts add an element, deletes remove one, and updates remove the old element and add the new one, so sketches follow a change stream without rescans.

IV. VERIFYING WHILE WRITES ARE IN FLIGHT

During a live migration T trails S by the replication lag, so a naive comparison reports every in-flight write. DELTAPROOF tags each sketch snapshot with a *watermark*, the highest source commit version it reflects, and compares S at the target's watermark w. Writes applied to T out of order, ahead of w, still appear as candidates, so each candidate is *confirmed* by re-reading both stores: equal values mean the write was in flight; if T is behind, the key is re-checked after a grace period Δ; a key that still differs is reported. Re-reads are point lookups on candidate keys only. In-flight writes enlarge the difference, so with write rate λ and lag L a level must hold about

$$m \approx c \cdot \frac{d_{\text{true}} + \lambda L}{P}, \qquad c \approx 2, \tag{2}$$

cells per partition; the strata estimator measures this combined size directly.

Section V proves when this protocol is exact and states precisely what confirmation guarantees.

V. CORRECTNESS

A. Model

Let $\mathcal{K}$ be the key space and $\mathcal{U} = \mathcal{K} \times \{0,1\}^{64}$ the element space. A store state is a partial map $\sigma : \mathcal{K} \rightharpoonup \mathcal{V}$; its element set is $E(\sigma) = \{(k, \delta(\sigma(k))) : k \in \text{dom}\,\sigma\}$ with $\delta = H_{64} \circ \text{canon}$. Two values are *equivalent*, $v \equiv v'$, if $\text{canon}(v) = \text{canon}(v')$; this is the equality the schema mapping declares (Section III-A). We model H_{64}, the cell-placement hashes $h_1, \ldots, h_q$ and the checksum g as independent random functions (the random-oracle model), and write $\varepsilon = 2^{-64}$.

A *signed multiset* is $f : \mathcal{U} \to \mathbb{Z}$ with finite support. A sketch with m cells is $\Phi(f) \in G^m$, $G = \mathbb{Z} \times \mathbb{F}_2^{128} \times \mathbb{F}_2^{64} \times \mathbb{F}_2^{64}$, where cell c holds

$$\Big(\textstyle\sum_{x \in c} f(x),\ \bigoplus_{x \in c} [f(x)]_2\, k(x),\ \bigoplus_{x \in c} [f(x)]_2\, \delta(x),\ \bigoplus_{x \in c} [f(x)]_2\, g(x) \Big),$$

PRELIMINARY DRAFT

"$x \in c$" means $h_j(x) = c$ for some j (the q subtables make the q cells of x distinct), and $[\cdot]_2$ is reduction mod 2. We write $\mathbf{1}_A$ for the indicator of a set A.

B. Sketches follow the data

Lemma 1 (Linearity): $\Phi(f + f') = \Phi(f) \boxplus \Phi(f')$, where $\boxplus$ adds counts and XORs the other fields.

Proof: Each field of each cell is the image of f under a group homomorphism: summation into $\mathbb{Z}$ for the count, and $f \mapsto \bigoplus [f(x)]_2 \, a(x)$ for the others, because $[\cdot]_2 : \mathbb{Z} \to \mathbb{F}_2$ is a homomorphism. A sum of homomorphisms is a homomorphism. ∎

Lemma 2 (Change-stream maintenance): Let σ_0 be the state at which the sketch is built and $c_1, c_2, \ldots$ the changes after it in commit order, where c_i on key k_i carries a before-image b_i and an after-image a_i (either may be absent). If every before-image is complete, $b_i = \sigma_{i-1}(k_i)$, then applying $-\mathbf{1}_{e(b_i)} + \mathbf{1}_{e(a_i)}$ for $i \le r$ to $\Phi(\mathbf{1}_{E(\sigma_0)})$ yields $\Phi(\mathbf{1}_{E(\sigma_r)})$, where $e(\cdot)$ is the element of a row and $e(\text{absent})$ contributes nothing.

Proof: Only key k_i changes at step i, so $\mathbf{1}_{E(\sigma_i)} = \mathbf{1}_{E(\sigma_{i-1})} - \mathbf{1}_{e(b_i)} + \mathbf{1}_{e(a_i)}$ because $b_i = \sigma_{i-1}(k_i)$. Induct on i and apply Lemma 1. ∎

In PostgreSQL, `REPLICA IDENTITY FULL` makes before-images complete. The same lemma covers the target sketch when its before-images come from the side table of last written digests, provided nothing writes to the target outside the pipeline.

C. Decoding is exact or reports failure

For states S, T let $\Delta = \mathbf{1}_{E(S)} - \mathbf{1}_{E(T)}$; then $\Delta(x) \in \{-1, 0, 1\}$, and by Lemma 1 the subtracted sketch is $\Phi(\Delta)$. Peeling accepts a cell when its count is ± 1 and its checksum equals $g(x^*)$ for the element x^* formed by its key and digest fields; it then subtracts $\pm\mathbf{1}_{\{x^*\}}$. It reports success when no cell is left nonzero.

Theorem 1 (Exact decoding): Let peeling of $\Phi(\Delta)$ examine N cells and output O. If it reports success, then $O = \Delta$ except with probability at most $(N+1)\varepsilon$. If it reports failure, the nonzero cells identify every partition whose elements may be incomplete.

Proof: Call an acceptance *false* if the accepted cell's residual content $R = \operatorname{supp}(\Delta - O) \cap c$ is not exactly $\{x^*\}$. Until the first false acceptance, the invariant "the table equals $\Phi(\Delta - O)$ and O agrees with Δ on $\operatorname{supp} O$" holds, because a truly pure cell yields $x^* \in \operatorname{supp} \Delta$ with the right sign. Before the first false acceptance, which cells are examined and in what order depends only on Δ and the placement hashes, so it is independent of g. A false acceptance at a cell needs $\bigoplus_{y \in R} g(y) = g(x^*)$ with $R \neq \{x^*\}$ and $R \neq \emptyset$ (an empty R has count 0). If $x^* \notin R$, then $g(x^*)$ is independent of $\{g(y)\}_{y \in R}$; otherwise $\bigoplus_{y \in R \setminus \{x^*\}} g(y) = 0$ for a nonempty set. Either way the probability is ε, and a union bound over the N examinations gives $N\varepsilon$. If peeling reports success but $\Delta' = \Delta - O \neq 0$, pick $x \in \operatorname{supp} \Delta'$ and a cell $c \ni x$: its checksum field $\bigoplus_{y \in R_c} g(y)$ is 0 with $x \in R_c$, which has probability ε. This adds the final ε. Cells are zero only when every partition decoded, so the nonzero cells name every incomplete partition. ∎

Corollary 1 (Correctness independent of sizing): Let $D(S,T) = \{k : S(k) \not\equiv T(k)\}$, where an absent key is equivalent only to an absent key. Verification returns $D(S,T)$, split into missing, extra and mismatched keys, except with probability at most $(N+1)\varepsilon + |D(S,T)|\varepsilon$, for any choice of level, partition count or estimate.

Proof: Partitions that decode contribute exactly their part of $\operatorname{supp} \Delta$ by Theorem 1; failed partitions are compared exactly by rescanning. Each store contributes at most one element per key, so for each key $\operatorname{supp} \Delta$ holds nothing, one element of either sign, or one of each sign with different digests; these are exactly "equal", "missing in T", "extra in T" and "value mismatch". A key with $S(k) \not\equiv T(k)$ is missed only if H_{64} maps the two different canonical values to the same digest, which happens with probability ε per such key. ∎

The strata estimate and the ladder level therefore affect cost, not correctness. The canonical words are bijections of the canonical values (SplitMix64 is invertible), so collisions arise only in H_{64} and in the 64-bit text hash, both covered by ε in the random-oracle model.

D. Verification while writes are in flight

Let source changes have versions in commit order, and let S_w be the source state after all changes with version $\le w$. The replicator *processes* each change: it applies it, applies it altered, or drops it. Its *watermark* w_T is the largest w such that every change with version $\le w$ has been processed.

Theorem 2 (Watermark exactness): Assume Lemma 2's conditions on both sides and *in-order application*: at the verification instant the target reflects the processing of exactly the changes with version $\le w_T$. Then verification of $\Phi(\mathbf{1}_{E(S_{w_T})})$ against the target sketch returns $D(S_{w_T}, T)$, the keys on which the pipeline's processing of the first w_T changes diverged, with the error bound of Corollary 1. No confirmation is needed.

Proof: By Lemma 2 the source sketch advanced to w_T is $\Phi(\mathbf{1}_{E(S_{w_T})})$, and the target sketch is $\Phi(\mathbf{1}_{E(T)})$ for the target's actual state. Apply Corollary 1 to (S_{w_T}, T). ∎

Our replicator satisfies in-order application: each batch is applied whole, with the sketch lock held, and verification runs between batches. Rescanned partitions compare current states rather than states at w_T, so their keys are candidates that need confirmation.

Proposition 1 (Out-of-order application): If the target reflects all changes with version $\le w_T$ plus a set A of later changes, then the output C satisfies $D(S_{w_T}, T) \setminus \operatorname{keys}(A) \subseteq C \subseteq D(S_{w_T}, T) \cup \operatorname{keys}(A)$.

Proof: For $k \notin \operatorname{keys}(A)$ the target value is that of in-order processing, so Theorem 2's argument applies key by key; keys in $\operatorname{keys}(A)$ may land on either side. ∎

Version-aware confirmation. Let each target row record the source version $v_T(k)$ of the change that wrote it. From the change stream, keep for each key k its state at the watermark, $(\lambda_w(k), \delta_w(k))$: the version of the last change to k with

PRELIMINARY DRAFT

version $\leq w_T$ (0 if none since the sketch was built) and the digest after it, or "absent". Also keep the list $\Lambda(k)$ of pairs (v, δ_v) for delivered changes to k with $v > w_T$. Let $V(k) = \{(\lambda_w(k), \delta_w(k))\} \cup \Lambda(k)$, the states the source held for k at or after w_T. Report a candidate k if and only if

$$\big(v_T(k),\ \delta(T(k))\big) \notin V(k),$$

where an absent target row matches any element of $V(k)$ whose digest is "absent". The rule reads only the target and the stream-fed index, never the source, and decides at once.

Theorem 3 (Sound and complete confirmation): Assume the target is written only by the pipeline. Up to H_{64} collisions: (i) if k is reported, the target holds for k no state that the source held at or after w_T; (ii) if the pipeline's last processed change to k has version at most w_T and was processed incorrectly (dropped, altered, or applied over a newer state), then k is reported.

Proof: (i) $(\lambda_w(k), \delta_w(k))$ is k's source state at version w_T, and each $(v, \delta_v) \in \Lambda(k)$ is its state just after change $v > w_T$. So $V(k)$ contains only states the source held at or after w_T, and a target pair outside it matches none of them. (ii) Let the faulty change c have version $v_c \leq w_T$, with no later change to k processed. Then $\lambda_w(k) \geq v_c$ and every pair in $\Lambda(k)$ has version $> w_T \geq v_c$. If c was dropped, or a stale state was written over it, the target holds a state written by an earlier change, with version $v' < v_c \leq \lambda_w(k)$; a deleted key whose delete was dropped still holds such a row. That version is in no pair of $V(k)$. If c was altered, the target holds (v_c, δ') with $\delta' \neq \delta_c$; when $v_c = \lambda_w(k)$ the digest differs from $\delta_w(k)$, and when $v_c < \lambda_w(k)$ the version matches no pair. In every case the target pair is not in $V(k)$, unless H_{64} collides (ε). ■

A fault on a change with version above w_T may still match (λ_w, δ_w), so it is decided at a later check, once w_T passes it. The sketch comparison supplies the candidates: under in-order application they are exactly the divergent keys (Theorem 2), and under out-of-order application a superset (Proposition 1), which the rule filters.

Current-read confirmation. A simpler rule needs no versions: re-read both stores at t_1 and again at $t_2 \geq t_1 + \Delta$, and report k if both reads differ.

Proposition 2: If every change is processed within time $L < \Delta$ of its commit, and no change to k commits in $[t_1 - L,\ t_2]$, then current-read confirmation reports k if and only if the pipeline processed k's last change incorrectly.

Proof: The last change to k committed before $t_1 - L$, so it was processed by $t_1 \leq t_2$. No newer change exists by t_2, so $S(k)$ at t_2 is that change's after-image, and $T(k)$ at t_2 is its processed result. Both reads therefore differ exactly when the processing was incorrect. ■

For keys written continually during the check, such as TPC-C's `district` rows, both reads can differ because of lag alone, so current-read confirmation can report a key that is only behind. The version-aware rule of Theorem 3 removes this case, and our live experiments use it.

VI. Implementation

DeltaProof is implemented in Python 3 with NumPy, about 1,100 lines for the library plus tests, benchmarks and adapters, released under the Apache 2.0 license [repository URL to be added]. Cells are 36 bytes on the wire (4-byte count, 16-byte key sum, 8-byte digest sum, 8-byte checksum). Hashing uses SplitMix64 mixing in bit-identical vectorized and scalar forms, and BLAKE2b-64 for text. The baselines share the encoder and a common cost model: bytes sent to the verifier, sequential round trips, and rows the stores read in bulk. Bisection is charged as if both stores could serve key-range checksums, its best case. Adapters read PostgreSQL through psycopg 3 and Cassandra through the DataStax driver.

VII. Evaluation

A. Setup

In-process experiments hold source and target as columnar arrays so that every method is measured under one cost model, with $n \in \{10^5, 10^6, 10^7\}$ entries of a synthetic double-entry ledger (Zipf-distributed accounts, $\theta = 0.99$; log-normal amounts in cents; five currencies) and d up to 112,500 differing keys. The end-to-end experiments use PostgreSQL 16.4 and Cassandra 4.1.6, single nodes in Docker, with the ledger, TPC-H `lineitem` and a live TPC-C workload. All runs used one Apple M1 Pro laptop (10 cores, 16 GB), Python 3.13 and NumPy 2.5, seed 42.

B. Detection (RQ1)

Each fault class was injected alone, 100 faults into 10^6 rows. DeltaProof, the full scan, bisection and Merkle trees found 100% of every class with no false positives. Per-partition count and sum aggregates detected lost writes, phantoms, stale values and lost deletes, but 84% of precision faults and none of the corruption, time-zone or offsetting faults.

C. Cost (RQ2)

Every method returned exactly the injected keys in all 14 configurations. Fig. 2 and Table III show the results at $n = 10^7$. DeltaProof read no table rows at verification time, while every baseline read about $2n$ rows or more. With no differences, Merkle trees (16 B) and bisection (1 KB) send less than DeltaProof's 187 KB estimator, but only after scanning both tables. Near 1% differing rows the advantage over a full scan shrinks to 12×.

PinSketch. We also compared PinSketch, BCH-syndrome reconciliation [11], [9], using the minisketch C++ library [10]. To be fair to it, PinSketch received DeltaProof's strata estimate and the same key partitioning, with each partition's capacity set to its expected load plus four standard deviations. It was exact in every configuration and is the stronger method on bandwidth: from 1,125 differences up, it sent 3.4–9.4× fewer bytes than DeltaProof (Table II), because a 64-bit syndrome per unit of capacity is smaller than a 36-byte IBLT cell at load 0.5. DeltaProof is the stronger method on computation. From 11,250 differences up it decoded 5–10× faster (below that PinSketch decoded faster, as its $O(c^2)$

PRELIMINARY DRAFT

TABLE II
DELTAPROOF VS. PINSKETCH AT $n = 10^7$

	MB sent		Decode (s)		PinSketch
d	DELTAPROOF	PinSketch	DELTAPROOF	PinSketch	build (s)
11	0.19	0.19	0.13	0.01	6.2
1,125	0.78	0.23	0.24	0.15	34.2
11,250	4.91	0.59	0.28	1.55	37.4
112,500	39.4	4.18	1.41	14.5	36.1

DELTAPROOF's ladder, built once for any d, took 18–20 s for both stores.

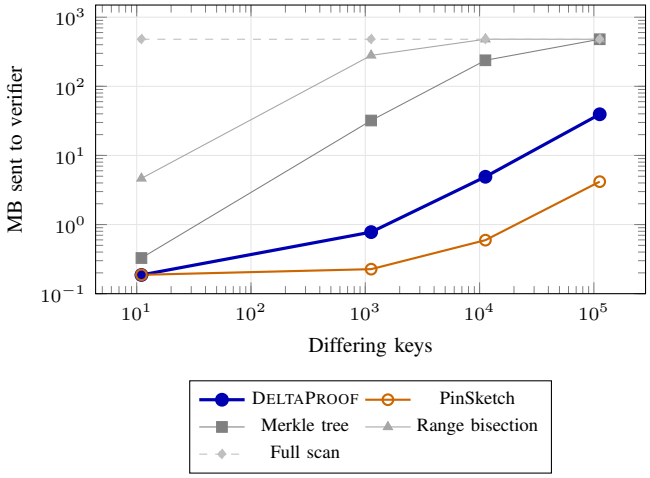


Fig. 2. Bytes sent to the verifier at $n = 10^7$ (log–log). DELTAPROOF's traffic grows with the number of differences; the baselines approach a full scan.

decoding is cheap at small capacity), and its whole ladder of six sizes was built for both stores in 18 s, in NumPy, while one PinSketch sized for a known estimate took 34–37 s in C++. Adding an element costs $q = 3$ cell updates per IBLT level but one update per unit of capacity in a PinSketch partition, which matters when sketches follow every write of a change stream. The encoding, partitioning, watermark and confirmation in this paper apply to either sketch; IBLTs suit high write rates and PinSketch suits constrained links.

D. Live writes (RQ3)

On 10^6 rows with 50 true faults and 50,000 in-flight writes (Table IV), the naive comparison reported up to 9,942 false candidates. Watermarks alone removed them all when the target applied writes in order, but not when 10–50% of later writes arrived early. With confirmation, every configuration reported exactly the 50 true faults and none falsely, at about two point reads per candidate.

E. Overhead (RQ4) and robustness (RQ5)

At $n = 10^7$ the sketches took 173 MB per store (17.3 bytes per row across six levels). Bulk loading cost 0.89 µs per row; batched updates of 2,000 writes cost 3.8 µs per write; single updates cost about 1.1 ms, almost all Python interpreter overhead. A partition sized for 256 differences in 513 cells failed to decode in 0.3% of trials at full load, 1.3% at 1.25×, 10% at 1.5× and always at 2×. Strata estimates had

TABLE III
VERIFICATION COST AT $n = 10^7$: MB SENT / MILLION ROWS READ / ROUND TRIPS

d	DELTAPROOF	Merkle	Bisection	Full scan
0	0.19 / 0 / 1	0.00 / 20.0 / 1	0.00 / 20.0 / 2	480 / 20.0 / 1
11	0.19 / 0 / 1	0.33 / 20.0 / 16	4.65 / 25.8 / 3	480 / 20.0 / 1
1,125	0.78 / 0 / 2	32.0 / 21.3 / 16	280 / 51.7 / 3	480 / 20.0 / 1
11,250	4.91 / 0 / 2	238 / 29.9 / 16	480 / 60.0 / 3	480 / 20.0 / 1
112,500	39.4 / 0.06 / 3	480 / 40.0 / 16	480 / 60.0 / 3	480 / 20.0 / 1

TABLE IV
FALSE CANDIDATES UNDER IN-FLIGHT WRITES ($n = 10^6$, 50 TRUE FAULTS)

Lag	Early	Naive	Watermark	+Confirm	Reads
100	0%	100	0	0	100
1,000	10%	882	118	0	336
10,000	0%	9,942	0	0	100
10,000	50%	4,890	5,071	0	10,280

median ratios of 0.98–1.00 to the true difference, with 90% of estimates between 0.81 and 1.17, which supports the 1.25× headroom. When a partition did fail, rescanning it cost at most 58,510 rows, 0.3% of the rows in both stores.

F. Real stores

We loaded 200,000 entries into PostgreSQL and Cassandra and injected 120 faults of six classes with SQL and CQL statements. Scanning took 0.47 s on PostgreSQL and 3.6 s on Cassandra; canonical encoding took 4.3 s and verification 12 ms. DELTAPROOF found all 120 faults with no false positives in two round trips and 260 KB. A digest of the drivers' raw values flagged 199,835 rows, 199,715 of them falsely, because the stores format and round timestamps differently.

G. TPC-H data

To test the encoding on a standard schema we did not design, we generated TPC-H `lineitem` at scale factor 1 (6,001,215 rows) with `dbgen` through DuckDB, loaded it into PostgreSQL and Cassandra, and injected 120 faults of six classes (lost writes, precision, date shifts, corruption, lost deletes, phantoms). These are not TPC-H benchmark results. The exact difference of full scans of both stores equalled the injected set, and DELTAPROOF found all 120 keys with no false positives in two round trips and 260 KB; verification took 0.13 s and building both sketches 11 s. A digest of the drivers' raw values flagged every row, because PostgreSQL returns `CHAR(n)` values padded and Cassandra's driver returns its own date type. Loading Cassandra took 750 s and scanning with row-by-row canonical encoding took 222 s on PostgreSQL and 397 s on Cassandra in Python; a sketch fed by a change stream avoids these scans.

H. Live TPC-C migration

We ran BenchBase [14] TPC-C (4 warehouses, 8 terminals) on PostgreSQL. The eight TPC-C tables with primary

PRELIMINARY DRAFT

TABLE V
LIVE TPC-C MIGRATION, POSTGRESQL → CASSANDRA

Rate (tx/s)	200	200	200	max
Lag; order	2 s	5 s	2 s; 5% late	2 s
Changes captured	359,083	358,665	364,449	1,569,177
Injected faults	330	332	344	1,510
Live checks	11	11	10	6
Candidates (sum)	1,500	1,528	40,622	813
Reported (sum)	1,500	1,528	1,380	581
False reports / missed	0 / 0	0 / 0	0 / 0	0 / 0
Median check time	0.37 s	0.43 s	1.85 s	1.38 s
True differences at end	248	266	275	988
Final answer exact	yes	yes	yes	yes
Sketches equal rebuilds	yes	yes	yes	yes
TPC-C p99, alone	7.3 ms	7.3 ms	7.3 ms	9.2 ms
TPC-C p99, with pipeline	19.9 ms	18.1 ms	22.0 ms	17.3 ms
TPC-C tx/s, alone / with	200 / 200	200 / 200	200 / 200	3,368 / 1,963

keys hold 1.98 million rows; `history` has no key and is excluded. Changes were read with the wal2json logical-decoding plugin [15], with full before-images. A replicator applied them to Cassandra after a fixed lag, recorded the source version in each row, and dropped 0.05% and corrupted 0.05% of changes. In the out-of-order run it also deferred 5% of changes by 0.5–1.5 s, keeping per-key order, so later changes to other keys landed first. The source sketch followed the change stream, and the target sketch followed the values the replicator wrote. Every 10 s, with the replicator paused between batches, DELTAPROOF compared the sketches at the target's watermark and confirmed candidates with Theorem 3's rule, which reads only the target. Each report was checked against what the replicator had actually done to that key's last processed change. At the end, both sketches were compared with sketches rebuilt from full scans, and the final answer with the exact difference of those scans.

Table V summarizes four runs. Across 38 live checks no report was false and no fault on a change at or below the watermark was missed. With in-order application the candidates equalled the reports at every check, as Theorem 2 predicts. With 5% of changes late, there were 27 times as many candidates as in the in-order run, as Proposition 1 allows, and the version-aware rule removed every in-flight key. In the unlimited-rate run the replicator's 99th-percentile apply delay reached 147 s with up to 610,000 queued changes, and verification stayed exact. In every run the sketches maintained from the change stream equalled full-scan rebuilds bit for bit, and the final answer equalled the true difference. Checks sent 187–778 KB with in-order application and up to 4.9 MB in the out-of-order run, whose in-flight keys enlarge the difference the sketches must hold (Section IV).

TPC-C's p99 latency at 200 tx/s rose from 7.3 ms alone to 18–22 ms with the pipeline running; an earlier run of the same configuration measured 7.9 ms. At an unlimited rate throughput fell 42%. All components shared one 10-core laptop, so these numbers mix the cost of logical decoding with CPU contention from the Python replicator, Cassandra and BenchBase, and we treat them as upper bounds. Sketch maintenance, including canonical encoding, cost 39–42 µs per captured change and 37–44 µs per applied write in Python.

VIII. DISCUSSION AND LIMITATIONS

With no differences, the 187 KB estimator costs more than a single root-hash comparison; a first round of one digest per store would close the gap. The live experiment ran every component on one laptop, so its source-overhead numbers at high load are an upper bound; separating the replicator onto its own machine is future work. The target sketch was maintained from the values the replicator wrote, so changes made to the target outside the pipeline would need the target's own change stream or periodic partition rescans to be seen. Single updates in Python are slow; batching change events reduces the cost to microseconds. Real-store tests used up to 6 million rows on single nodes. Tables without a primary key cannot be verified key by key. A wrong schema mapping could hide a mismatch if both sides are mis-encoded the same way, so mappings need their own tests.

IX. RELATED WORK

Set reconciliation. Minsky *et al.* [11] reconcile sets with near-optimal communication using characteristic polynomials, and PinSketch [9], implemented in minisketch [10], does so with BCH syndromes; we compare against it in Section VII; IBLTs [6] with strata estimators [7] made it practical, and rateless designs [8] remove the need to estimate d. These target network synchronization; we add cross-store canonical encoding, CDC-fed maintenance and verification under live writes. *Replica anti-entropy.* Cassandra repairs replicas with Merkle trees [4], [5]; practitioners have suggested IBLTs could improve full-table repair [16]. *Migration verification.* data-diff [3] bisects key-range checksums across databases. *Online migration.* F1's online schema change [2] and industry reports on dual writes [1] address how to migrate, not how to prove the result; Kleppmann [12] surveys derived data and Helland [13] motivates idempotent, entity-scoped processing.

X. CONCLUSION

Verifying a live heterogeneous migration is a set reconciliation problem. Treating it that way makes verification cost follow the number of differences instead of the data size. At 10^7 rows and about 1,100 differences, DELTAPROOF sent $41\times$ fewer bytes than the next-best method while reading no table rows, and it found every injected fault, with none reported falsely, on real PostgreSQL and Cassandra stores.

ACKNOWLEDGMENT

During the preparation of this manuscript the authors used Claude (Anthropic) to assist with drafting and editing the text of all sections, writing the prototype implementation and benchmark scripts, and producing the figures and tables. All experiments were executed by running that code; every reported number comes from the resulting CSV files. The authors reviewed and take full responsibility for all content, including the verification of every citation.

PRELIMINARY DRAFT

References

[1] J. Xu, "Online migrations at scale," Stripe Engineering Blog, Feb. 2017. [Online]. Available: https://stripe.com/blog/online-migrations

[2] I. Rae *et al.*, "Online, asynchronous schema change in F1," *Proc. VLDB Endow.*, vol. 6, no. 11, pp. 1045–1056, 2013.

[3] data-diff, "Technical explanation," project documentation. [Online]. Available: https://data-diff.readthedocs.io/en/latest/technical-explanation.html

[4] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *ACM SIGOPS Oper. Syst. Rev.*, vol. 44, no. 2, pp. 35–40, 2010.

[5] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology—CRYPTO '87*, LNCS vol. 293, 1988, pp. 369–378.

[6] M. T. Goodrich and M. Mitzenmacher, "Invertible Bloom lookup tables," in *Proc. 49th Annu. Allerton Conf. Commun., Control, Comput.*, 2011, pp. 792–799.

[7] D. Eppstein, M. T. Goodrich, F. Uyeda, and G. Varghese, "What's the difference? Efficient set reconciliation without prior context," in *Proc. ACM SIGCOMM*, 2011, pp. 218–229.

[8] L. Yang, Y. Gilad, and M. Alizadeh, "Practical rateless set reconciliation," in *Proc. ACM SIGCOMM*, 2024.

[9] Y. Dodis, R. Ostrovsky, L. Reyzin, and A. Smith, "Fuzzy extractors: How to generate strong keys from biometrics and other noisy data," *SIAM J. Comput.*, vol. 38, no. 1, pp. 97–139, 2008.

[10] P. Wuille, G. Maxwell, and G. Naumenko, "minisketch: An optimized library for BCH-based set reconciliation," GitHub repository. [Online]. Available: https://github.com/bitcoin-core/minisketch

[11] Y. Minsky, A. Trachtenberg, and R. Zippel, "Set reconciliation with nearly optimal communication complexity," *IEEE Trans. Inf. Theory*, vol. 49, no. 9, pp. 2213–2218, 2003.

[12] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[13] P. Helland, "Life beyond distributed transactions: An apostate's opinion," in *Proc. CIDR*, 2007, pp. 132–141.

[14] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudré-Mauroux, "OLTP-Bench: An extensible testbed for benchmarking relational databases," *Proc. VLDB Endow.*, vol. 7, no. 4, pp. 277–288, 2013.

[15] E. Taveira, "wal2json: JSON output plugin for changeset extraction," GitHub repository. [Online]. Available: https://github.com/eulerto/wal2json

[16] ScyllaDB Community Forum, "Recent discussion of invertible Bloom lookup tables at Monster Scale Summit 2026," 2026. [Online]. Available: https://forum.scylladb.com/t/recent-discussion-of-invertible-bloom-lookup-tables-at-monster-scale-summit-2026/5352

PRELIMINARY DRAFT